

Deep Learning — Interview Answers

Easy · Medium · Hard — Professional Answers with Real-World FANG Examples

Professional Answers · Real-World FANG Examples · All Difficulty Levels

EASY — 24 Questions with Answers

Q1. What is a neural network? Describe how a forward pass works.

A neural network is a computational model loosely inspired by biological neurons, organised into layers. Each neuron computes a weighted sum of its inputs, adds a bias, and passes the result through an activation function. A forward pass propagates data from input to output: (1) input layer receives the raw features. (2) Each hidden layer computes: $\text{output} = \text{activation}(W \times \text{input} + b)$, where W is the weight matrix, b is the bias vector. (3) The output layer produces predictions. Each layer transforms its input into a new representation, progressively extracting higher-level features. The output is compared to the true label using a loss function to measure error.

Real-World Example:

At Google, a 3-layer neural network classifies search query intent. Forward pass for 'Python tutorial': input layer receives a 512-dimensional embedding vector. Hidden layer 1 (1024 neurons, ReLU activation): detects features like 'programming language reference' vs 'snake biology'. Hidden layer 2 (512 neurons): detects more abstract patterns like 'learning intent'. Output layer (3 classes, softmax): produces $[0.87, 0.09, 0.04]$ for [informational, navigational, transactional]. The forward pass takes $<1\text{ms}$ on CPU. Google runs this for 8.5 billion searches per day, requiring massive parallelism across thousands of inference servers.

Q2. What is backpropagation?

Backpropagation is the algorithm that computes gradients of the loss function with respect to every weight in the network. It uses the chain rule of calculus to propagate the error signal backwards through the network, from the output layer to the input layer. Steps: (1) Forward pass computes the output and loss. (2) Compute $dL/d(\text{output})$ — gradient of loss w.r.t. the final layer output. (3) Using the chain rule: $dL/dW_k = dL/d(\text{next_layer_input}) \times d(\text{next_layer_input})/dW_k$. This 'backpropagates' through each layer. (4) Each weight's gradient is computed in $O(n)$ total time for a network with n parameters.

Real-World Example:

At Meta training a 65B parameter LLM: backpropagation computes gradients for 65 billion weights in one backward pass. The chain rule traces through 96 transformer layers. A subtle practical challenge: gradient vanishing — gradients near the input layers can become exponentially small after multiplying through 96 layers, making early layers learn very slowly. Fixes: residual connections (skip connections add the gradient directly to the input, bypassing multiplication), layer normalisation (stabilises gradient scale). Meta's mixed-precision training computes forward pass in float16 (2× faster) but accumulates gradients in float32 (prevents gradient underflow). Without backpropagation's exact gradient computation, 65B parameter training would be infeasible.

Q3. What is the vanishing gradient problem? What activation functions help?

The vanishing gradient problem occurs when gradients become exponentially small as they are backpropagated through many layers, because gradients are multiplied by the derivative of each activation function at every layer. If the derivative is consistently < 1 , gradients decrease exponentially with depth. For the sigmoid function, the derivative is at most 0.25 — gradients halve at each layer, so through 20 layers: $0.25^{20} \approx 10^{-12}$ (effectively zero). Early layers stop learning. Solutions: ReLU (derivative = 1 for positive inputs — no vanishing for active neurons). Residual connections (skip connections carry the gradient directly). Layer normalisation and batch normalisation stabilise gradient scale.

Real-World Example:

At Google, the original deep CNNs for image recognition (pre-2015) suffered from vanishing gradients beyond 8-10 layers — training accuracy plateaued and deeper models were no worse than shallower ones. The introduction of ReLU (Krizhevsky's AlexNet, 2012) directly addressed this: ReLU's derivative is exactly 1 for positive inputs, not 0.25. Training depth jumped from 8 layers (with sigmoid) to 22 layers (GoogLeNet, 2014). ResNet (2015) introduced skip connections, enabling 152+ layers. The ResNet paper's key result: a 152-layer network outperforms a 34-layer network on ImageNet — impossible with sigmoid (vanishing gradients) but achievable with ReLU + skip connections.

Q4. What is dropout and why does it prevent overfitting?

Dropout randomly zeroes a fraction p (0.1-0.5) of neuron outputs during each training forward pass. Each neuron is dropped independently with probability p . During inference, all neurons are active but their outputs are scaled by $(1-p)$ to maintain the same expected output magnitude. Dropout prevents overfitting because: (1) it forces the network to learn redundant representations — no single neuron can rely on specific other neurons being present. (2) It can be interpreted as training an ensemble of 2^n different thinned networks (one per dropout mask) and averaging their predictions at test time. (3) It adds noise to the gradient updates, acting as a regulariser similar to L2.

Real-World Example:

At Hinton's lab (University of Toronto, now at Google), dropout was introduced in 2012 (Srivastava et al.). The original dropout paper on MNIST: a 3-layer network without dropout achieved 1.6% test error; with dropout ($p=0.5$): 1.05% — a 34% improvement. At Meta's image classification models, dropout ($p=0.1-0.3$) in the final fully-connected layers prevents memorisation of training set idiosyncrasies. For transformer models (GPT, BERT), dropout is applied to attention weights and FFN outputs. However, very large language models (GPT-4 scale) use dropout=0.0 — the regularisation from the massive dataset size and weight decay is sufficient, and dropout slows training without benefit at >100B parameter scale.

Q5. What is batch normalisation and what problem does it solve?

Batch normalisation (BatchNorm) normalises the outputs of a layer within each mini-batch to have zero mean and unit variance, then applies learned scale (gamma) and shift (beta) parameters. For each feature d : $x_{\text{norm}_d} = (x_d - \text{mean_batch}_d) / \sqrt{\text{var_batch}_d + \epsilon}$. Output: $\gamma_d \times x_{\text{norm}_d} + \beta_d$. Benefits: (1) Reduces internal covariate shift — as earlier layers update their weights, the distribution of inputs to later layers changes, requiring later layers to constantly re-adapt. BatchNorm stabilises these distributions. (2) Enables higher learning rates by preventing gradients from exploding or vanishing due to extreme activation values. (3) Acts as a regulariser — the mini-batch statistics add noise, reducing overfitting slightly.

Real-World Example:

At Google, batch normalisation was introduced by Ioffe and Szegedy (2015) and reduced ImageNet training time from 90 to 14 epochs while achieving better accuracy. VGG (without BN) needed learning rate 0.01; GoogLeNet with BN used learning rate 0.1 (10× higher). The key insight: without BN, using $lr=0.1$ causes gradient explosion because layer outputs have large variance that amplifies during backpropagation. BN keeps layer outputs normalised, making the loss landscape smoother and enabling aggressive learning rates. Modern vision models (ResNet, EfficientNet) universally use BatchNorm. Transformers instead use LayerNorm (normalise across features for one sample, not across samples) because batch size can be small or 1 in sequence models.

Q6. What is the difference between a CNN and a fully connected network for images?

A fully connected (dense) network for images treats each pixel as an independent input feature — no spatial structure is exploited. For a $224 \times 224 \times 3$ image: 150,528 input features, and the first hidden layer with 1000 neurons requires $150,528 \times 1000 = 150\text{M}$ parameters — just for the first layer. A CNN (Convolutional Neural Network) uses filters/kernels: a $3 \times 3 \times 3$ filter with 64 output channels requires only $3 \times 3 \times 3 \times 64 = 1,728$ parameters and is applied across the entire image (weight sharing). This exploits two inductive biases: (1) Translation invariance — a cat in the top-left looks the same as a cat in the bottom-right, so the same filter should detect it. (2) Local structure — pixels near each other are more related than pixels far apart.

Real-World Example:

At Google's image search (2012 revolution): AlexNet (CNN with 5 convolutional layers) used 60M parameters and achieved 15.3% top-5 error on ImageNet. A comparably deep fully connected network would require trillions of parameters for the same depth — infeasible. The CNN's weight sharing is the key: the same 3×3 edge detector filter is applied at every spatial position, so learning 'detect vertical edges' once generalises everywhere. At Google Photos, their CNN detects faces, scenes, and objects. Without CNNs, fully connected networks would need $10,000 \times$ more parameters for image tasks, making training and inference impractical for 6 billion photos uploaded daily.

Q7. What is transfer learning? Why is it so powerful?

Transfer learning takes a model trained on a large source task (typically ImageNet with 1.2M images for vision, or a large text corpus for NLP) and fine-tunes it on a smaller target task. The pre-trained model has already learned general features (edges, textures, shapes for images; grammar, semantics for text) in its early layers. Fine-tuning: either (1) freeze early layers, only train the final classification head (few parameters to tune, fastest, best for very small datasets), or (2) fine-tune all layers with a small learning rate (allows all features to specialise to the target domain, better for larger datasets). Transfer learning dramatically reduces the data needed for the target task.

Real-World Example:

At Stanford Medical School, their pneumonia detection model used transfer learning from ImageNet. Training from scratch on 1,000 chest X-rays: model failed to learn useful features (too little data). Transfer learning from ImageNet (frozen convolutional layers, new classification head): AUC=0.90 in 2 days of training on the same 1,000 X-rays. The ImageNet features (edge detectors, texture detectors) transferred effectively to X-ray texture patterns indicating pneumonia. Fine-tuning all layers with $lr=1e-5$: AUC=0.94 — only 1,000 labelled X-rays needed. Training from scratch would require ~100,000 X-rays for the same AUC. Transfer learning made expert-level pneumonia detection achievable without hospital-scale labelled datasets.

Q8. What is the attention mechanism in deep learning?

Attention allows a model to focus on relevant parts of the input when computing each output element, rather than processing all parts equally. For each output position, attention computes a weighted sum of all input representations, where weights reflect how relevant each input position is to the current output position. Query-Key-Value formulation: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$. Query (Q) represents what the current position is looking for. Key (K) represents what each input position contains. Value (V) is the actual content to mix. The dot product QK^T measures compatibility; softmax converts to probabilities (attention weights); V is weighted-mixed according to those weights.

Real-World Example:

At Google, attention was transformative for machine translation. In 2014, the simple encoder-decoder RNN bottlenecked all meaning into one fixed-size vector. Attention added a mechanism: when translating 'the bank of the river', the decoder could attend strongly to 'river' when choosing the correct translation of 'bank' (riverbank vs financial bank). Attention weights are visualisable: the attention weight matrix for a German translation showed the decoder attended to the corresponding German words while producing each English word. In 2017, the Transformer architecture made attention the primary mechanism (replacing RNNs entirely), powering BERT, GPT, PaLM, and all modern LLMs. The innovation: attention enables each word to 'see' every other word in the sequence, capturing long-range dependencies that RNNs struggle with.

Q9. What is overfitting in deep learning? Name three ways to address it.

Overfitting in deep learning occurs when a model has too many parameters relative to the training data size — it memorises training examples rather than learning generalisable patterns. Deep neural networks have millions of parameters and can trivially memorise small datasets. Signs: training loss decreasing while validation loss increases. Methods to address: (1) Dropout — randomly zero activations during training, preventing co-adaptation. (2) Data augmentation — artificially expand the training set by applying label-preserving transformations (for images: flipping, rotation, cropping, colour jitter). (3) Weight decay (L2 regularisation) — add $\lambda \times \sum(w^2)$ to the loss, penalising large weights and encouraging simpler models. Additional methods: early stopping, reducing model capacity, mixup augmentation.

Real-World Example:

At Google's image classification: training a ResNet-50 on 50,000 custom product images (vs ImageNet's 1.2M) showed severe overfitting — training accuracy 97%, validation accuracy 74%. Three interventions applied: (1) Dropout ($p=0.3$) in the final FC layers: validation improved to 79%. (2) Data augmentation (random horizontal flip, 10-degree rotation, colour jitter): validation improved to 84%. (3) L2 weight decay ($\lambda=1e-4$): validation improved to 86%. Combining all three: 89% validation accuracy with training accuracy at 91% (small gap — good generalisation). The 15-point improvement from 74% to 89% validated all three regularisation techniques independently contributing.

Q10. What is the sigmoid activation function? When is it used?

The sigmoid function: $\sigma(x) = 1 / (1 + e^{-x})$. Maps any real number to (0, 1). Output can be interpreted as a probability. Derivative: $\sigma(x) \times (1 - \sigma(x))$, maximum value 0.25 at $x=0$. Use cases: binary classification output layer (output = probability of positive class). Attention gates (output gates in LSTMs). NOT recommended for hidden layers in deep networks due to vanishing gradient (max derivative 0.25, compounded through layers becomes tiny) and gradient saturation (for very large/small inputs, derivative ≈ 0 — neuron is 'dead'). Alternatives for hidden layers: ReLU (faster, no vanishing), GELU (smoother, preferred in transformers), Swish.

Real-World Example:

At Airbnb, the booking probability model's output layer uses sigmoid: $P(\text{booking}) = \text{sigmoid}(W \times \text{features} + b)$. The output is directly interpretable as a probability (0.73 = 73% chance of booking), which is used in pricing algorithms: higher booking probability $\rightarrow$ lower surge factor to encourage bookings. For the 32 hidden layers inside the model, Airbnb uses ReLU (not sigmoid) to avoid vanishing gradients. The sigmoid output layer is essential for probability calibration — if they used ReLU on the output (range 0 to infinity), the output could not be interpreted as a probability and would require an additional calibration step.

Q11. What is the ReLU activation function? What are its advantages?

ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$. Output is the input if positive, zero if negative. Derivative: 1 if $x > 0$, 0 if $x < 0$ (undefined at $x=0$, treated as 0 in practice). Advantages: (1) No vanishing gradient for positive inputs — gradient flows through unchanged. (2) Computationally very cheap — just a threshold operation, no exponentials. (3) Sparsity — roughly half the neurons output zero for any input, making the representation sparse and the network more efficient. (4) Practical performance — consistently outperforms sigmoid and tanh in deep networks. Disadvantages: dying ReLU — neurons with negative pre-activations always output 0 and never update (gradient=0). Fix: Leaky ReLU ($f(x) = \max(0.01x, x)$) or ELU.

Real-World Example:

AlexNet (2012) was the first major deep CNN to use ReLU instead of sigmoid and tanh. The paper reported that ReLU was 6× faster to reach 25% training error compared to tanh on CIFAR-10. At Google, ReLU reduced training time for their image classifier from 3 weeks to 5 days (4× speedup) when switching from sigmoid to ReLU hidden layers — the same network, same data, same hardware. Google's benchmarks showed the dominant cost was the $\exp()$ computation in sigmoid; ReLU's $\max()$ operation is 100× faster on GPU hardware. GELU (used in BERT, GPT) is a smooth approximation of ReLU that performs slightly better on NLP tasks: $\text{GELU}(x) = x \times P(X \leq x)$ where X is standard normal.

Q12. What is a loss function? Give three examples with their use cases.

A loss function measures the discrepancy between the model's predictions and the true labels. It is minimised during training via gradient descent. The choice of loss function determines what 'good predictions' means and directly shapes what the model learns. Loss functions must be differentiable (to enable gradient computation). Three examples: (1) Cross-entropy loss (classification): $L = -\sum y_i \log(p_i)$. Minimising cross-entropy maximises the probability assigned to the correct class. Used for classification tasks. (2) MSE (regression): $L = \text{mean}((y - \hat{y})^2)$. Penalises large errors quadratically. Used when large deviations are very costly. (3) Binary cross-entropy (binary classification): $L = -(y \log(p) + (1-y) \log(1-p))$. The standard loss for sigmoid output layers.

Real-World Example:

At Uber, three different models use three different loss functions: (1) Ride request surge pricing (regression): MSE loss — a \$5 pricing error matters less than a \$50 error, and MSE's quadratic penalty correctly weights large errors. (2) Fraud detection (binary classification): Binary cross-entropy with class weights — fraud rate is 0.1%, so positive examples get weight 1000 in the loss to balance the gradient contribution. (3) Driver activity classification (multi-class: available, busy, offline): Categorical cross-entropy — each driver is in exactly one state. Uber also uses focal loss for extreme imbalance (in the fraud detector when the positive rate is < 0.01%) — focal loss $(1-p)^\gamma \times \log(p)$ down-weights easy negatives, focusing training on hard examples.

Q13. What is the difference between a CNN and an RNN?

CNN (Convolutional Neural Network): processes data with spatial or local structure (images, audio spectrograms) using sliding filter operations. Each filter captures a local pattern across the entire input (weight sharing). Parallel computation — all positions processed simultaneously. No memory of previous inputs. Best for: image recognition, object detection, audio classification. **RNN (Recurrent Neural Network):** processes sequential data by maintaining a hidden state that is updated at each time step: $h_t = \tanh(W_h \times h_{t-1} + W_x \times x_t)$. The hidden state carries 'memory' of previous inputs. Sequential computation — cannot parallelise across time steps. Best for: time series, speech, language modelling (though now largely replaced by Transformers).

Real-World Example:

At Apple, both architectures co-exist in Siri. CNN: acoustic model that converts raw audio spectrogram to phoneme probabilities. The spectrogram is treated like an image — temporal and frequency axes are spatial dimensions. CNNs detect patterns like 'fricative consonant' or 'vowel' in local windows. RNN (LSTM): language model that predicts the most likely word sequence given the phoneme probabilities and linguistic context. The LSTM maintains context about the current sentence structure, disambiguating 'recognise speech' vs 'wreck a nice beach' (same phonemes, different context). Apple's Siri pipeline: audio → CNN acoustic model → phoneme probabilities → LSTM language model → text. This sequential combination is the standard ASR (automatic speech recognition) architecture.

Q14. What is a Transformer architecture? Why did it replace RNNs?

The Transformer (Vaswani et al., 2017) consists entirely of attention mechanisms and feed-forward layers — no recurrence. Core components: multi-head self-attention (each token attends to all other tokens simultaneously), feed-forward networks (applied independently to each position), residual connections, and layer normalisation. Replaced RNNs because: (1) **Parallelisation:** RNNs process tokens sequentially (token 10 must wait for token 9 to finish) — Transformers process all tokens simultaneously — massive GPU utilisation improvement. (2) **Long-range dependencies:** RNNs struggle to maintain context over 500+ tokens; Transformers directly connect any two positions via attention, enabling perfect long-range recall. (3) **Scaling:** Transformer performance scales smoothly with model size and data, enabling the GPT/BERT/PaLM generation.

Real-World Example:

At Google, the switch from RNN to Transformer for Google Translate (GNMT to Transformer) in 2017 improved BLEU score by 4 points while reducing training time from 6 days to 2 days on the same hardware. The speedup came from GPU parallelism: RNN processes a 100-word sentence in 100 sequential steps (100% sequential); Transformer processes it in 1 parallel step (100% parallel on GPU) — a 50-100× GPU utilisation improvement. Long-range dependency improvement: the RNN's hidden state forgot content from 200+ tokens ago; the Transformer's attention mechanism can copy information from position 1 to position 500 with zero degradation. This enabled accurate translation of long legal documents where references span paragraphs.

Q15. What is word2vec? How does it learn word embeddings?

Word2vec learns dense vector representations of words (embeddings) by training a shallow neural network to predict word context. Two architectures: Skip-gram: given a center word, predict the surrounding context words. CBOW: given context words, predict the center word. Training objective: maximise the probability of true context words while minimising probability of random (noise) words — negative sampling makes this tractable. The model learns embeddings such that words appearing in similar contexts have similar vectors. Result: semantic meaning encoded in geometry — king – man + woman $\approx$ queen; Paris – France + Germany $\approx$ Berlin (analogy relationships captured).

Real-World Example:

At Google Research (Mikolov et al., 2013), word2vec training on 100B words from Google News produced 300-dimensional embeddings with remarkable properties. The famous king-queen analogy emerges because: 'king' and 'queen' appear in similar regal contexts, while 'man' and 'woman' appear in similar gender contexts — the vector offset captures the gender relationship. At Airbnb, word2vec-style training on 'listing browsing sequences' creates listing embeddings: listings that are frequently viewed together in the same browsing session appear as similar vectors. This embedding approach (treat browsing sessions like sentences, listings like words) enabled Airbnb's 'similar listings' feature to understand that '2-bedroom apartment in Paris near Eiffel Tower' is semantically similar to '2-bedroom apartment in Paris near Louvre' even with no shared words.

Q16. What is a GAN (Generative Adversarial Network)?

A GAN consists of two networks trained in opposition: the Generator (G) creates fake data from random noise, trying to make it look real. The Discriminator (D) tries to distinguish real data from fake data generated by G. Training: D is trained to output 1 for real, 0 for fake. G is trained to make D output 1 for its fakes (fool D). This adversarial game leads to G generating increasingly realistic data and D becoming increasingly good at detecting fakes. At Nash equilibrium, G produces data indistinguishable from real data and D outputs 0.5 for everything (maximum uncertainty). Loss functions: D minimises $L_D = -\log(D(\text{real})) - \log(1-D(G(z)))$. G minimises $L_G = -\log(D(G(z)))$ (wants D to say its fakes are real).

Real-World Example:

At NVIDIA, StyleGAN (2019) generates photorealistic human faces that don't exist. Training on 70,000 high-quality celebrity photos, StyleGAN produces 1024×1024 images indistinguishable from real photos — the D failed to do better than random at distinguishing them. Practical applications: Adobe Photoshop's 'Generative Fill' uses a GAN to fill in missing image regions realistically. Netflix uses GANs to generate additional training data for their content moderation classifier — generating synthetic examples of edge-case harmful content that is rare in the real dataset, without needing to collect or expose humans to real harmful content. The synthetic examples improve the classifier's recall on rare categories by 15%.

Q17. What is a variational autoencoder (VAE)?

A VAE is a generative model that learns to compress data into a probabilistic latent space, then decode from that space back to data. Encoder: maps input $x \rightarrow$ parameters of a distribution (mean μ and log-variance σ) in the latent space. The latent code z is SAMPLED from $N(\mu, \sigma^2)$. Decoder: reconstructs x from z . The VAE loss = reconstruction loss (how well x is reconstructed) + KL divergence (how close the posterior $q(z|x)$ is to the prior $N(0,1)$). The reparameterisation trick: $z = \mu + \sigma \times \text{epsilon}$ (epsilon $\sim N(0,1)$) makes the sampling differentiable. Unlike standard autoencoders, the continuous, structured latent space enables generating new data by sampling $z \sim N(0,1)$ and decoding.

Real-World Example:

At Spotify, a VAE learns a continuous 'music style space'. Given a song, the encoder produces a distribution in style space — nearby points correspond to similar-sounding music. New music is generated by: (1) sampling a latent vector $z \sim N(0,1)$ and decoding — creates novel music in an average style, (2) interpolating between two songs' latent codes — creates a smooth musical transition between styles. Spotify uses VAE latent representations for playlist continuity: the similarity between consecutive songs is measured in the VAE latent space (not raw features), capturing perceptual similarity. VAE outperformed standard autoencoders for interpolation because the VAE's structured latent space (forced to match $N(0,1)$) ensures smooth transitions — standard AE latent spaces have gaps where decoded samples are nonsensical.

Q18. What is fine-tuning vs training from scratch?

Training from scratch: randomly initialise all weights and train on the target dataset from the beginning. Requires large amounts of data and compute. Suitable when: the target domain is very different from available pre-trained models, or enormous task-specific data is available. Fine-tuning: start from a pre-trained model's weights, then continue training on the target task. The pre-trained weights encode general features (ImageNet features for vision, language understanding for NLP). Fine-tuning adapts these to the target domain with much less data and compute. Strategies: (1) Full fine-tuning — update all weights (best for large target datasets). (2) Head-only fine-tuning — freeze the backbone, only train the new classification head (best for very small target datasets). (3) Layer-wise learning rate decay — lower learning rates for earlier layers (they need less adjustment) and higher for later layers.

Real-World Example:

At Microsoft Bing, fine-tuning BERT for search query understanding vs training from scratch: training from scratch on Bing's 100M query-document pairs took 6 weeks on 128 GPUs and achieved NDCG@10=0.87. Fine-tuning from pre-trained BERT (Google's bert-large-uncased, trained on 3.3B words) took 3 days on 8 GPUs and achieved NDCG@10=0.89 — better accuracy with 16× less compute. The BERT pre-training had already learned that 'jaguar' relates to both the animal and the car, that pronouns refer to entities, and that question words indicate search intent — all crucial for query understanding. Microsoft estimates pre-trained models save \$50M+/year in compute costs across their Azure ML services by eliminating from-scratch training for most applications.

Q19. What are activation functions and why are they necessary?

An activation function introduces non-linearity into the neural network after each linear transformation ($W \times \text{input} + b$). Without activation functions: a network of any depth is equivalent to a single linear transformation (compositions of linear functions are linear). With activation functions: the network can represent any continuous function (universal approximation theorem). The non-linearity allows the network to separate non-linearly separable classes, model complex relationships (XOR cannot be solved with a linear classifier), and learn hierarchical features (edges → textures → shapes → objects). Common activations: ReLU ($f(x)=\max(0,x)$), sigmoid (0 to 1), tanh (-1 to 1), GELU (smooth ReLU, used in Transformers), softmax (output layer for multiclass).

Real-World Example:

The XOR problem classically demonstrates why activation functions (non-linearity) are necessary. A 1-layer linear network cannot separate the XOR truth table (linear decision boundary). A 2-layer network with ReLU activation can: the first layer transforms the input space so the XOR classes become linearly separable, then the second layer draws the linear decision boundary. At Google's ad click prediction model: without non-linear activations, the model could only capture linear relationships between features (e.g. 'user age × ad category' interactions would be missed). With deep ReLU networks, Google's ad model captures thousands of non-linear feature interactions that explain a 15% CTR improvement over linear models.

Q20. What is an embedding layer in neural networks?

An embedding layer maps discrete tokens (words, item IDs, user IDs) to continuous dense vectors. For a vocabulary of size V with embedding dimension d : the embedding matrix is a $V \times d$ weight matrix, and looking up the embedding for token i is equivalent to selecting row i . This is a learnable lookup table. Embeddings encode semantic relationships: tokens that appear in similar contexts learn similar vectors. Embedding layers dramatically reduce the dimensionality compared to one-hot encoding (V dimensions $\rightarrow$ d dimensions, typically $d=64$ to 1024 , vs V which can be $100,000+$). Embeddings are the input to virtually all NLP and recommendation models.

Real-World Example:

At Netflix, their recommendation model uses 3 types of embeddings: (1) User embedding (512-dimensional) captures taste profile — two users who both love sci-fi documentaries have similar embeddings. (2) Movie embedding (512-dimensional) captures content characteristics — a movie embedding encodes genre, style, director, actors as a dense vector. (3) Genre tag embedding (64-dimensional). The recommendation score for a (user, movie) pair is the dot product of their embeddings. With 250M users $\times$ 512-d embeddings: the user embedding table is $250M \times 512 \times 4$ bytes = 512GB — too large for one machine. Netflix shards the embedding table across 64 GPUs. The embedding lookup (retrieve a row from the 512GB table) is the primary bottleneck in their recommendation model inference.

Q21. What is learning rate scheduling? Name three common schedules.

Learning rate scheduling adjusts the learning rate during training. A fixed learning rate is rarely optimal: too high causes training instability; too low wastes compute with very small updates. Three common schedules: (1) Step decay: reduce lr by a factor (0.1) every N epochs. Simple, widely used. (2) Cosine annealing: lr decreases following a cosine curve from lr_{max} to lr_{min} . Smooth, allows the model to escape sharp minima. Often combined with warm restarts. (3) Linear warmup + cosine decay: standard for large Transformer models. Start with $lr=0$ (or very small), linearly increase for the first N steps (warm-up prevents unstable large updates on random initial weights), then cosine decay to near-zero. This schedule is used in BERT, GPT, and most LLMs.

Real-World Example:

At Google, the learning rate schedule dramatically impacts model quality and training stability. Training BERT without warmup: with $lr=1e-4$ from step 1, training loss exploded in the first 500 steps (initial random weights produce large gradients). With 10K step linear warmup: stable training from the start. The cosine decay phase: after warmup, the lr decreases from $1e-4$ to $1e-6$ over 1M steps following a cosine curve. Compared to fixed $lr=1e-4$ for the same 1M steps: cosine decay improved BERT's fine-tuning accuracy on GLUE benchmark by 0.8% on average across 8 tasks — a consistent improvement that motivated Google to make warmup+cosine-decay the standard for all transformer training runs.

Q22. What is the difference between classification and generation tasks in deep learning?

Classification: given an input, produce a label from a fixed finite set. Output layer: softmax over K classes. Loss: cross-entropy. Model outputs a probability distribution over classes. No new data is created. Examples: image classification, sentiment analysis, intent detection. Generation: produce new data of the same type as training data — images, text, audio, code. Output: sequence of tokens or pixels, often conditioned on an input. The model must learn the full data distribution, not just a decision boundary. Much harder. Examples: GPT (text generation), DALL-E (image generation), AlphaCode (code generation), Whisper (speech transcription).

Real-World Example:

At OpenAI, GPT models are pure generation models — they predict the next token given all previous tokens. GPT-4 generates text by sampling from the learned distribution $P(\text{token} \mid \text{context})$. For classification tasks, GPT-4 frames them as generation: 'Sentiment of this review: Positive/Negative. Review: [text]. Sentiment:' — GPT generates either 'Positive' or 'Negative'. This unification is the insight behind instruction-tuned LLMs (GPT, Claude) — classification, translation, summarisation, and open-ended generation all become instances of the same next-token prediction task. OpenAI's research showed that GPT-4's 'generation-as-classification' achieved better zero-shot classification performance on many benchmarks than purpose-built classification models, because the generative pre-training on 1T+ tokens learned deeper semantic understanding.

Q23. What is the purpose of the softmax function in neural networks?

Softmax converts a vector of real-valued scores (logits) into a probability distribution that sums to 1. Formula: $\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$. Properties: all outputs are in (0,1); they sum to exactly 1. Exponential amplifies differences — larger logit → much larger probability (winner-takes-all tendency). Used in: multiclass classification output layers (convert logits to class probabilities), attention mechanisms (normalise attention scores to weights summing to 1). The temperature parameter T: $\text{softmax}(z/T)$ controls sharpness. $T \rightarrow 0$: argmax (one-hot), $T=1$: standard softmax, $T \rightarrow \infty$: uniform distribution (maximum entropy). Temperature scaling is used in knowledge distillation and controlling LLM output creativity.

Real-World Example:

At Google's neural machine translation, the softmax output layer converts the decoder's final hidden state into a probability distribution over the 32,000-word vocabulary. For each generated word position, the model produces 32,000 logits, softmax normalises them to probabilities, and the next word is sampled (or argmax'd). Temperature controls generation diversity: Google's translation system uses $T=1.0$ (standard) for accuracy. For creative writing suggestions in Google Docs, a higher temperature ($T=0.8-1.2$) produces more diverse word choices. For code completion in GitHub Copilot, a lower temperature ($T=0.2-0.4$) produces more deterministic, correct code completions. The same softmax layer with different temperatures serves different downstream applications.

Q24. What is the universal approximation theorem?

The universal approximation theorem states that a neural network with at least one hidden layer and a non-linear activation function can approximate any continuous function on a compact domain to any desired degree of accuracy, given enough neurons. Formally: for any continuous function f on $[0,1]^n$ and any $\epsilon > 0$, there exists a one-hidden-layer network with finitely many neurons that approximates f within ϵ at every point. The theorem guarantees expressiveness: neural networks are not limited to linear or polynomial relationships. However, it does not say how many neurons are needed (could be exponentially many for one layer), how to find the right weights (training problem), or how many training examples are needed (sample complexity).

Real-World Example:

At DeepMind, the universal approximation theorem motivated the hypothesis that a neural network could solve Go at superhuman level — Go's board evaluation function is a continuous function of the board state, so a neural network can in principle represent it. AlphaGo demonstrated this empirically: a network with 13 convolutional layers approximated both the value function (probability of winning from a given board state) and the policy function (probability distribution over next moves) well enough to surpass human grandmasters. The theorem's practical limitation was evident: the 'in principle' approximation required a network with millions of parameters and millions of training games — the theorem guarantees existence but not efficiency.

MEDIUM — 20 Questions with Answers

Q1. Implement a multi-head attention mechanism from scratch in PyTorch.

Multi-head attention allows the model to attend to information from different representation subspaces simultaneously. Split Q, K, V into h heads (d_{model}/h dimensions each). Run scaled dot-product attention independently in each head. Concatenate outputs. Apply final linear projection. Scaled dot-product: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$. Multi-head: $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$ where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$. Why multiple heads: different heads learn different types of relationships — one head might attend to syntactic relationships, another to semantic, another to positional. The $\sqrt{d_k}$ scaling prevents the dot products from growing too large and pushing softmax into saturation regions.

Real-World Example:

At Google, BERT uses 12 attention heads $\times$ 12 layers = 144 attention matrices. Visualisation research (Vig & Belinkov 2019) showed different heads learned different patterns: some heads attend to the next token (positional), some attend to matching brackets in code, some attend to the object of the current verb. At Tesla's Autopilot, multi-head attention in their Vision Transformer (ViT) processes 8 camera images simultaneously: different attention heads focus on different road features — one head attends to lane markings, another to vehicle bounding boxes, another to traffic lights. The modular specialisation of multiple heads is what gives Transformers their representational power.

Q2. Implement ResNet's residual block and explain why skip connections help.

Residual block: $\text{output} = F(x) + x$, where $F(x)$ is the block's main path ($\text{conv} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{conv} \rightarrow \text{BN}$) and x is the skip connection (identity shortcut). The block learns $F(x) = \text{desired_output} - x$ (the residual). If the desired output is close to x , $F(x) \approx 0$ — the block learns near-identity transformations easily (just set all weights to near-zero). Why skip connections help: (1) Gradient highway — the skip connection creates a direct path for gradients from output to input, bypassing the non-linear transformations. $dL/dx = dL/d(\text{output}) \times (1 + dF/dx)$. The +1 from the skip connection ensures gradients cannot vanish even if dF/dx is small. (2) Easier optimisation — the block can learn identity mappings as a trivial solution when the block's contribution is near-zero.

Real-World Example:

At Microsoft Research (He et al., 2015), ResNet demonstrated that skip connections enabled training networks 10-100 $\times$ deeper than previously possible. ResNet-18 (18 layers): top-5 error 27.9%. ResNet-152 (152 layers with skip connections): top-5 error 19.4%. VGG-19 (19 layers, no skip connections): top-5 error 24.4%. Without skip connections, training 152 layers produced higher error than 18 layers due to the degradation problem (not overfitting — vanishing gradients made deep layers useless). With skip connections, adding depth consistently improved accuracy. The paper showed: accuracy on a 56-layer plain network was WORSE than a 20-layer plain network on CIFAR-10 (training error, not just test error — confirming the problem was optimisation difficulty, not overfitting). ResNets solved this completely.

Q3. Implement a word2vec Skip-gram model with negative sampling.

Skip-gram: given a center word c , predict context words within a window. Negative sampling objective: maximise $\log(\sigma(\mathbf{v}_c \cdot \mathbf{u}_o)) + k \times E_j[\log(\sigma(-\mathbf{v}_c \cdot \mathbf{u}_j))]$, where o is a true context word, j are k noise/negative samples. The model maintains two embedding matrices: target embeddings W ($V \times d$) and context embeddings C ($V \times d$). Training loop: for each center-context pair (c, o) : sample k negative words. compute target embedding $\mathbf{v}_c = W[c]$. compute true context embedding $\mathbf{u}_o = C[o]$. compute k negative embeddings $\mathbf{u}_j = C[j]$. Loss: $-\log(\sigma(\mathbf{v}_c \cdot \mathbf{u}_o)) - \sum \log(\sigma(-\mathbf{v}_c \cdot \mathbf{u}_j))$. Backward pass: update $W[c]$, $C[o]$, and $C[j_{1..k}]$ only (sparse updates — much faster than updating the full vocabulary).

Real-World Example:

At Pinterest, word2vec-style training on user pin sequences learns 'PinSage' embeddings. Each pin (image + description) is treated as a 'word'; each user's pinning session is treated as a 'sentence'. Skip-gram with negative sampling trains embeddings such that pins frequently co-pinned by the same user have similar vectors. The negative sampling is crucial: without it, the softmax denominator requires computing dot products with all 3 billion pins (infeasible). With $k=5$ negative samples, each update touches 7 pins (1 positive + 5 negative + 1 center) instead of 3 billion. Pinterest's PinSage system serves billions of 'Related Pins' recommendations per day. The embedding similarity in the 512-dimensional PinSage space captures aesthetic and thematic similarity that raw image features miss.

Q4. Explain the BERT pre-training objectives and fine-tuning procedure.

BERT (Bidirectional Encoder Representations from Transformers) uses two pre-training objectives: (1) Masked Language Modelling (MLM): randomly mask 15% of tokens. 80% of masked tokens are replaced with [MASK], 10% with a random word, 10% left unchanged. The model must predict the original token. This forces bidirectional context learning — unlike GPT which only sees left context. (2) Next Sentence Prediction (NSP): given sentence A and sentence B, predict if B actually follows A. Helps model understand sentence relationships. Fine-tuning: add a task-specific classification head on top of the [CLS] token embedding. Train all layers with a small learning rate ($2e-5$ to $5e-5$) for 3-5 epochs. Fine-tune the full model end-to-end. Different tasks require different heads: linear classifier for classification, span extraction for QA, token classifier for NER.

Real-World Example:

At Google, BERT was the first model to achieve human-level performance on GLUE (General Language Understanding Evaluation benchmark) in 2018. On the SQuAD 2.0 question answering task: previous SOTA (ensemble model) achieved $F1=78.0\%$; BERT-large achieved $F1=87.4\%$ — a 9-point improvement in one model. The bidirectional training was the key differentiator: for sentiment analysis of 'I love not being unhappy', the [MASK] on 'unhappy' must use both left context ('not being') and right context (end of sentence) to correctly predict 'unhappy' rather than 'happy'. Unidirectional LMs (GPT-style) can only use left context. Fine-tuning BERT for Google Search query understanding (adding a relevance classification head) improved search quality by 10% on conversational queries — the biggest search quality improvement in 5 years at the time.

Q5. Implement a simple Transformer encoder block.

Encoder block components: (1) Multi-head self-attention: $x_{\text{attn}} = \text{MultiHeadAttention}(Q=x, K=x, V=x, \text{mask}=\text{None})$. Allows each position to attend to all others. (2) Add + Norm (post-attention residual): $x = \text{LayerNorm}(x + x_{\text{attn}})$. (3) Feed-forward network: $x_{\text{ff}} = \text{FFN}(x) = \text{Linear}(\text{ReLU}(\text{Linear}(x)))$. Typical dimensions: $d_{\text{model}} \rightarrow 4 \times d_{\text{model}} \rightarrow d_{\text{model}}$. (4) Add + Norm (post-FFN residual): $x = \text{LayerNorm}(x + x_{\text{ff}})$. Pre-norm variant (more stable training): apply LayerNorm before each sub-layer: $x = x + \text{Attn}(\text{LayerNorm}(x))$; $x = x + \text{FFN}(\text{LayerNorm}(x))$. GPT-2/3 uses pre-norm; original Transformer uses post-norm. The encoder processes ALL tokens in parallel (no mask needed, unlike the decoder which masks future tokens).

Real-World Example:

At Meta (PaLM, LLaMA architecture), each Transformer encoder block is the fundamental computational unit. LLaMA-2's architecture details: $d_{\text{model}}=4096$, 32 attention heads (each 128-dimensional), FFN with SwiGLU activation (not ReLU) and 11,008-dimensional intermediate: $d_{\text{ff}} = 8/3 \times d_{\text{model}} (\approx 2.67 \times \text{ instead of } 4 \times)$. RMSNorm instead of LayerNorm (faster, similar performance). The FFN accounts for 2/3 of a Transformer block's parameters (and compute). Meta's ablation: replacing $4 \times d_{\text{model}}$ FFN with $2.67 \times d_{\text{model}}$ SwiGLU FFN improved perplexity by 1.3 points at the same parameter count — the smoother activation function (SwiGLU) was the key factor.

Q6. Design and implement a neural collaborative filtering recommendation system.

Neural Collaborative Filtering (NCF) combines GMF (Generalised Matrix Factorisation) and MLP paths. GMF path: element-wise product of user and item embeddings $\rightarrow$ linear layer. MLP path: concatenate user and item embeddings $\rightarrow$ multiple fully connected ReLU layers. NeuMF: concatenate GMF and MLP outputs $\rightarrow$ final sigmoid output layer predicting interaction probability. Training: implicit feedback (clicked/not-clicked). Negative sampling: for each positive interaction, sample $k=4$ random unobserved items as negatives. Loss: binary cross-entropy over positive and negative samples. Evaluation: Hit Rate@K and NDCG@K — for each user, rank all unobserved items by predicted score; HR@10 = fraction of users for whom the held-out item is in the top 10.

Real-World Example:

At Netflix, Neural Collaborative Filtering outperformed pure matrix factorisation (ALS) on their A/B test: MF trained on the Netflix Prize dataset (100M ratings) achieved NDCG@10=0.73; NCF achieved 0.81 — a 10.9% improvement. The MLP path captured non-linear user-item interactions that the bilinear GMF path missed. For example, a user who only watches French films and a French film: the inner product of their embeddings was not high (the model hadn't learned the French preference from ratings alone), but the MLP path learned to detect the nationality feature interaction. Netflix's production recommendation uses a significantly more complex NCF variant with content features (genre, actors, director) as additional input alongside collaborative signals.

Q7. Implement a complete image classification pipeline with data augmentation and validation.

Full pipeline: Dataset class (load images, apply transforms). Data augmentation (training only): RandomHorizontalFlip, RandomCrop, ColorJitter, Normalize. Validation transforms: Resize, CenterCrop, Normalize only (no augmentation — we want to measure true performance). DataLoader: training=shuffle=True, num_workers=4 (parallel loading). Model: ResNet-18 with modified final layer (output_features=num_classes). Optimiser: Adam with lr=1e-3, weight_decay=1e-4. Loss: CrossEntropyLoss. Training loop: forward pass, compute loss, backward, optimiser step, zero gradients. Validation loop: no_grad() context, track accuracy. Learning rate scheduler: ReduceLROnPlateau (reduce lr when val_loss stops improving). Early stopping: stop training if val_accuracy doesn't improve for 5 epochs.

Real-World Example:

At Google Photos, every new classifier update goes through a standardised pipeline. Data augmentation choices: RandomHorizontalFlip adds 2× effective data with zero annotation cost. ColorJitter is critical for food photos (the same dish looks different under different lighting). MixUp augmentation (blend two images and interpolate labels) improved food classification accuracy from 87% to 92%. The validation pipeline is strictly no-augmentation: augmenting validation data inflates apparent accuracy (augmented crops are easier than full images seen in production). Google discovered that test-time augmentation (applying augmentation to test images and averaging predictions) improved production accuracy by 1.5% — effectively using augmentation for inference without test set contamination.

Q8. Explain and implement KL divergence and its role in VAEs.

KL divergence (Kullback-Leibler divergence) measures how much one probability distribution P differs from a reference distribution Q : $KL(P||Q) = \sum P(x) \log(P(x)/Q(x))$. It is not symmetric: $KL(P||Q) \neq KL(Q||P)$. $KL=0$ iff $P=Q$. $KL \geq 0$ always (Gibbs inequality). In VAEs: the encoder learns a posterior $q(z|x) = N(\mu(x), \sigma(x)^2)$. The prior is $N(0, I)$. The KL term in the ELBO loss: $KL(q(z|x) || p(z)) = 0.5 \times \sum (\mu^2 + \sigma^2 - \log(\sigma^2) - 1)$. This term forces the posterior to stay close to the prior. Without KL: each input encodes to a different isolated region of latent space — no smooth interpolation possible. With KL: the latent space is continuous and well-structured — nearby points in latent space decode to similar outputs.

Real-World Example:

At OpenAI, the VAE architecture underlies the DALL-E image generation system. The VAE's KL term creates a structured latent space where 'a photo of a cat' and 'a painting of a cat' have nearby latent representations, enabling interpolation between photographic and artistic styles. OpenAI's VQ-VAE (Vector-Quantised VAE) discretises the latent space (512 codebook entries $\times$ 32 $\times$ 32 spatial grid) to create a compact image representation that GPT-2 can autoregressively model. The original VAE's KL term was replaced with a commitment loss in VQ-VAE because the discrete codebook inherently prevents posterior collapse. The VQ-VAE encodes a 256 $\times$ 256 image into a 32 $\times$ 32 grid of codebook indices — a 64× compression that preserves visual quality well enough for DALL-E to generate realistic images from these codes.

Q9. What is knowledge distillation? Implement teacher-student training.

Knowledge distillation (Hinton et al., 2015) trains a smaller student model to mimic the soft output probabilities of a larger teacher model, rather than learning from hard labels. Teacher: a large, pre-trained model with high accuracy. Student: a smaller, more efficient model. Soft labels: teacher's output probabilities (with temperature $T > 1$ to reveal the probability mass on non-top classes). Loss = $\alpha \times \text{KL}(\text{student_soft}, \text{teacher_soft}) + (1-\alpha) \times \text{CrossEntropy}(\text{student}, \text{hard_labels})$. The soft labels contain 'dark knowledge': e.g., for an image labelled 'car', the teacher might assign 0.01 probability to 'truck' — this tells the student that cars and trucks are similar, providing richer training signal than the binary 0/1 label.

Real-World Example:

At Google, the Neural Architecture Search (NAS) for MobileNet used distillation: teacher = ResNet-50 (25.6M parameters), student = MobileNetV3-Small (2.5M parameters, 10× smaller). Without distillation: MobileNetV3-Small achieves 67.4% ImageNet top-1 accuracy. With distillation from ResNet-50: 68.9% — a 1.5% improvement despite the student being 10× smaller than the teacher. The temperature $T=4$ was used: teacher softmax(logits/4) spreads probability over multiple classes, revealing class similarity structure. The student learns not just 'this is a cat' but 'this is cat-like, slightly dog-like, with some fox-like features.' Google uses distillation in production to serve lightweight models on mobile devices while maintaining accuracy close to their large server-side models.

Q10. Implement class activation mapping (CAM) for visualising CNN decisions.

Class Activation Mapping uses the global average pooled feature maps from the last convolutional layer to visualise which spatial regions a CNN focuses on for a classification decision. For a CNN with last conv layer F (C channels, $H \times W$ spatial), followed by GAP (global average pooling) to C features, followed by a linear layer W (K classes $\times C$). For class c : $\text{class_score} = W[c] \cdot \text{GAP}(F) = \sum_k W[c,k] \times \text{GAP}(F_k)$. Class activation map $M_c = \sum_k W[c,k] \times F_k$ ($H \times W$). Upsample M_c to image size and overlay. High values indicate image regions most responsible for the class prediction. Requires GAP before the classification head — original CAM is not applicable to architectures with fully-connected layers (use GradCAM instead, which is architecture-agnostic).

Real-World Example:

At Stanford Medical School, CAM visualisations validated their chest X-ray pneumonia detection model. The radiologist review process: model predicts 'pneumonia' with 91% confidence; CAM overlaid on the X-ray shows high activation over the lower-right lung region. The radiologist confirms: 'Yes, I see the infiltrate in the right lower lobe.' This validation was required before clinical deployment — a model that achieves high accuracy by using artefacts (e.g. the hospital logo in the corner of X-rays from hospitals that only treat pneumonia patients) would show high CAM activation on the logo, not the lung. For 3 of 10 test X-rays, the CAM showed activation in anatomically implausible locations — revealing dataset bias before the model was deployed to patients.

Q11. Implement beam search decoding for sequence generation.

Beam search maintains K candidate sequences (beams) at each step, expanding each with every possible next token, scoring by log-probability sum, and keeping only the top-K sequences. Algorithm: initialise beams = [(0.0, [BOS])]. For each step: expand: for each beam (score, seq): for each token t in vocabulary: new_seq = seq + [t]; new_score = score + log_prob(t | seq). Prune: keep the K beams with highest new_score. Stop: if all top-K beams end with EOS or max_length reached. Return the top-1 (or top-K) completed sequences. K=1 is greedy decoding. Larger K: better quality but K× slower and K× more memory. Length penalty: divide score by length^{alpha} (alpha=0.6-0.8) to prevent short sequences from winning.

Real-World Example:

At Google Translate, beam search with K=4 and length penalty alpha=0.6 is the default decoding strategy. Greedy decoding (K=1) on English→German translation: BLEU=35.1. Beam search K=4: BLEU=37.3. K=10: BLEU=37.8. K=20: BLEU=37.9. The diminishing returns beyond K=4: the marginal quality gain of considering 5-20 beams is tiny compared to the 4× computational cost. Google measured the wall-clock impact: K=4 adds 3ms latency vs K=1 (2× computational cost, but I/O dominates). K=10 adds 9ms. The BLEU improvement from K=1 to K=4 (2.2 points) corresponds to roughly 3 months of feature engineering on the model side — making beam search one of the highest ROI decoding choices. For very latency-sensitive applications (real-time voice translation), K=1 greedy decoding is used.

Q12. What is the LSTM and how does it solve the vanishing gradient problem?

LSTM (Long Short-Term Memory) adds a cell state c_t alongside the hidden state h_t . The cell state acts as a 'conveyor belt' — information can flow through many time steps with minimal modification (only additive updates, no multiplicative squashing). Gates control information flow: Forget gate $f_t = \sigma(W_f \times [h_{t-1}, x_t] + b_f)$ — which cell state to erase. Input gate $i_t = \sigma(W_i \times [h_{t-1}, x_t] + b_i)$ — which new information to write. Cell candidate $\tilde{c}_t = \tanh(W_c \times [h_{t-1}, x_t] + b_c)$ — new candidate values. Cell update: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. Output gate $o_t = \sigma(W_o \times [h_{t-1}, x_t] + b_o)$. Hidden state: $h_t = o_t \odot \tanh(c_t)$. Vanishing gradient solution: the gradient flows through c_t via only addition and element-wise multiplication by gates — no repeated matrix multiplication that causes exponential decay.

Real-World Example:

At Google's speech recognition (pre-Transformer era), LSTM-based RNNs processed 10-second audio clips (250 time steps at 40ms frames). The critical long-range dependency: 'bank' in 'I went to the bank near the river' vs 'I withdrew from the bank' — the correct pronunciation depends on context 5-10 words earlier. Vanilla RNNs forgot context beyond 20-30 steps; LSTMs reliably maintained context for 100+ steps via the cell state. Google's 2016 LAS (Listen, Attend and Spell) LSTM model achieved 5.6% WER on LibriSpeech, vs vanilla RNN's 23.4% on the same data — demonstrating the practical importance of long-range memory for speech. Modern Transformers have largely replaced LSTMs for speech, but LSTMs remain deployed in resource-constrained environments where Transformer's $O(n^2)$ attention is too expensive.

Q13. Implement positional encoding for Transformers and explain why it's needed.

Transformers have no inherent notion of token order (self-attention treats all positions symmetrically). Without positional encoding, 'dog bites man' and 'man bites dog' produce identical attention patterns. Fix: add a positional embedding to each token embedding. Sinusoidal PE (original Transformer): $PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$; $PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_model)})$. Properties: unique encoding per position, smooth interpolation, and positions can be extrapolated beyond training length (because the functions are defined for all pos, not just training positions). Alternative: learned absolute positional embeddings (BERT, GPT-2) — each position gets a trainable vector. Modern alternative: RoPE (Rotary Position Embedding, LLaMA) — encodes position in the attention computation rather than additively, better extrapolation.

Real-World Example:

At Meta, LLaMA uses RoPE (Rotary Position Embeddings) instead of sinusoidal PE. RoPE encodes relative positions: the attention score between positions i and j encodes their distance ($j-i$) via rotation in embedding space. This allows LLaMA-2 to extrapolate to longer sequences than seen during training (trained on 4K context length, extrapolated to 32K with context interpolation). BERT's learned absolute PEs fail completely beyond training length (position 513 has no embedding if trained on 512 tokens). Sinusoidal PE extrapolates poorly beyond $\sim 2\times$ training length. Meta's RoPE implementation: rotate the Q and K vectors by position-dependent angles before computing attention scores — adds negligible compute overhead ($<0.1\%$ of total FLOP) while dramatically improving long-context performance.

Q14. What is contrastive learning and how is it used in CLIP?

Contrastive learning trains representations by pulling similar pairs closer and pushing dissimilar pairs apart in embedding space. The InfoNCE loss: $L = -\log(\exp(\text{sim}(z_i, z_j)/T) / \sum_k \exp(\text{sim}(z_i, z_k)/T))$ — maximise similarity of positive pair (i, j) relative to all negative pairs in the batch. CLIP (Contrastive Language-Image Pre-training, OpenAI 2021): image encoder (ViT or ResNet) and text encoder (Transformer) are trained with contrastive loss on 400M (image, text caption) pairs from the internet. Objective: for a batch of N pairs, the N correct (image, caption) pairs should have higher cosine similarity than the $N^2 - N$ incorrect pairs. At convergence, the image and text embeddings live in a shared semantic space: an image of a dog and the text 'a photo of a dog' have near-identical embeddings.

Real-World Example:

At OpenAI, CLIP enabled zero-shot image classification — classifying images into categories the model was never explicitly trained to classify. For ImageNet (1000 classes), CLIP's zero-shot accuracy: 76.2% top-1 accuracy, matching a supervised ResNet-50 (76.1%) without any training on ImageNet labels. The mechanism: for each class, create text prompts ('a photo of a {class}'). Encode all prompts. For a test image, find the class whose text embedding has highest cosine similarity with the image embedding. DALL-E 2 uses CLIP's text-image embedding space: to generate an image from text, it finds the CLIP image embedding that would correspond to the text description, then decodes that embedding to a pixel image (diffusion model). CLIP's shared embedding space is why DALL-E understands complex text prompts combining objects, styles, and scenes.

Q15. What is the scaled dot-product attention formula? Derive why scaling by $\sqrt{d_k}$ is necessary.

Attention(Q, K, V) = $\text{softmax}(QK^T / \sqrt{d_k}) \times V$. Derivation of $\sqrt{d_k}$ scaling: Q and K are d_k -dimensional vectors with elements drawn from approximately $N(0,1)$ after layer normalisation. The dot product $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ is a sum of d_k products of independent $N(0,1)$ random variables. Each product $q_i k_i$ has variance 1 ($\text{Var}[XY] = \text{Var}[X]\text{Var}[Y]$ if independent = $1 \times 1 = 1$ when $X, Y \sim N(0,1)$). The sum of d_k such terms has variance d_k (by independence). So $q \cdot k \sim N(0, d_k)$. Without scaling, for $d_k=64$, the dot products have standard deviation 8 — softmax input values range from -24 to +24, pushing softmax into saturation (extremely sharp attention weights). Scaling by $1/\sqrt{d_k}$ normalises the variance to 1, keeping softmax in its stable gradient region.

Real-World Example:

At Google Brain, the original Transformer paper's ablation experiments measured attention entropy (diversity of attention weights) with and without $\sqrt{d_k}$ scaling. Without scaling ($d_k=64$): attention entropy dropped to near-zero by layer 5 — the model was attending to at most 2-3 tokens and ignoring the rest. Effectively, multi-head attention collapsed to argmax attention (winner-takes-all). With scaling: attention entropy remained high throughout all 12 layers — the model maintained soft, distributed attention across many tokens. The quality impact: on the WMT14 English-German translation task, unscaled attention achieved BLEU=26.1; scaled attention achieved BLEU=27.3. The 1.2 BLEU improvement from this one formula change demonstrates how theoretically-motivated scaling decisions translate directly to empirical performance.

Q16. What are normalisation techniques? Compare BatchNorm, LayerNorm, and RMSNorm.

Batch Normalisation: normalise over the batch dimension for each feature. Stats computed over the mini-batch: μ_B = mean over batch, σ^2_B = variance over batch. Effective for CNNs where batch statistics are stable. Problems: behaves differently at train vs inference (uses running statistics at inference), batch size dependency (fails for batch size 1), and problematic for variable-length sequences. Layer Normalisation: normalise over the feature dimension for each sample independently. Stats computed over all features of one sample: μ = mean over features, σ^2 = variance over features. Batch size independent — works for batch size 1. Standard for Transformers. RMSNorm: simplification of LayerNorm — normalise by RMS only (no mean subtraction): $x / \text{RMS}(x) \times \gamma$. 7-15% faster than LayerNorm (no mean computation). Used in LLaMA, Mistral, most modern LLMs.

Real-World Example:

At Meta, comparing LayerNorm vs RMSNorm for LLaMA: on a 13B parameter model, replacing LayerNorm with RMSNorm reduced training step time by 8% (no mean subtraction = fewer operations per LayerNorm layer $\times$ 96 layers $\times$ every forward and backward pass). The perplexity difference: LLaMA-13B with LayerNorm = 4.12 perplexity; with RMSNorm = 4.10 (marginal improvement). Meta's conclusion: RMSNorm provides a free 8% training speedup with equal or better quality — all subsequent Meta LLMs use RMSNorm. BatchNorm remains standard for all CNN-based vision models (EfficientNet, ConvNeXt) where batch statistics are well-defined. The choice of normalisation is architecture-dependent: CNNs $\rightarrow$ BatchNorm, Transformers $\rightarrow$ LayerNorm or RMSNorm.

Q17. Implement gradient clipping and explain why it prevents training instability.

Gradient clipping limits the magnitude of gradients before the optimiser update step. Two methods: (1) Clip by value: if $\text{grad} > \text{max_value}$: $\text{grad} = \text{max_value}$ (per element). (2) Clip by norm: compute global gradient norm $G = \sqrt{\sum \text{grad}_i^2}$. If $G > \text{max_norm}$: scale all gradients by $\text{max_norm}/G$. Clip by norm is preferred: it preserves gradient direction while limiting magnitude. Implementation: after `loss.backward()`, before `optimizer.step()`:
`torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`. Training instability without clipping: in early training, random weight initialisation can produce extreme activations. Combined with large learning rate, this produces gradients with norms 10-1000× the typical gradient, causing catastrophic weight updates that throw the model parameters far from any good solution.

Real-World Example:

At OpenAI, gradient clipping ($\text{max_norm}=1.0$) is used in all GPT training runs. The instability event it prevents: 'loss spike'. During GPT-3 training, approximately 10 times during the 300B token training run, the loss suddenly jumped from the trend (a loss spike). Each time, gradient norms spiked to 100-1000× the typical value just before the loss spike. With gradient clipping: the gradient norm is limited to 1.0 regardless of its true magnitude — the worst-case update size is bounded. Without clipping: a single bad batch with extreme gradients could corrupt 10 hours of prior training progress. OpenAI's monitoring dashboard shows gradient norms throughout training; sustained high gradient norms ($>5\times$ normal) after clipping indicate a structural instability requiring architecture or hyperparameter changes.

Q18. What is the difference between autoregressive and masked language models?

Autoregressive LMs (GPT family): trained to predict each token given all PREVIOUS tokens (left-to-right context only). Training objective: maximise $P(x_t | x_1, \dots, x_{t-1})$ for all t . During generation: sample tokens left to right, each conditioned on all previously generated tokens. Advantages: natural for generation — the model directly generates text token by token. Masked LMs (BERT family): trained to predict MASKED tokens given BIDIRECTIONAL context (both left and right context). Training objective: maximise $P(x_t | x_1, \dots, x_{t-1}, x_{t+1}, \dots, x_n)$ for masked positions. Cannot directly generate text (would need to decide which tokens to mask). Advantages: better understanding representations for classification tasks — each token can attend to full context.

Real-World Example:

At Google, BERT (masked LM) achieved better performance on understanding tasks (question answering, sentiment analysis, NLI): GLUE score 80.5 vs GPT-1's (autoregressive) 72.8 — because BERT's bidirectional context is richer for classification. GPT (autoregressive) dominates generation tasks: BERT cannot generate coherent text because it was not trained to predict next tokens. The research community proposed T5 (encoder-decoder, combines both) and BART (bidirectional encoder + autoregressive decoder) as hybrids. Modern practice: fine-tune BERT for classification tasks, fine-tune GPT (or instruction-tune LLaMA) for generation and chat tasks. Google's PaLM and OpenAI's GPT-4 are purely autoregressive — the industry shifted toward autoregressive models because instruction tuning allows them to handle classification tasks via generation (generate 'positive' or 'negative') while remaining naturally suited for generation.

Q19. What is parameter-efficient fine-tuning (PEFT)? Explain LoRA.

PEFT methods fine-tune a fraction of a pre-trained model's parameters to adapt it to a new task, dramatically reducing the cost of fine-tuning large models. Full fine-tuning of a 70B model requires storing and computing gradients for 70B parameters (~280GB in float32). LoRA (Low-Rank Adaptation): for each weight matrix W ($d \times k$), add a low-rank update $\Delta W = BA$ where B ($d \times r$) and A ($r \times k$) with rank $r \ll \min(d, k)$. During fine-tuning: freeze W , only train B and A . Inference: $W_{\text{eff}} = W + BA$ (can be pre-merged, no inference overhead). Parameter reduction: instead of fine-tuning $d \times k = 4096 \times 4096 = 16.7\text{M}$ parameters per weight matrix, LoRA with $r=16$ trains $4096 \times 16 + 16 \times 4096 = 131\text{K}$ parameters — a $128\times$ reduction. Why low rank works: fine-tuning updates are often inherently low-rank — they modify only a few directions in weight space.

Real-World Example:

At Meta, LoRA fine-tuning of LLaMA-65B for instruction following: full fine-tuning requires 8 A100 80GB GPUs (260GB model) + 130GB gradient storage = 16 A100s minimum. LoRA with $r=16$: freeze 65B parameters, train only 0.3% (200M parameters) LoRA adapters — fits on 2 A100s. Quality comparison on Alpaca instruction following benchmark: full fine-tuning scores 71.4%; LoRA ($r=16$) scores 68.9% — only 2.5% quality drop for an $8\times$ GPU reduction. Meta researchers found that applying LoRA to all attention matrices (Q, K, V, O projections) and FFN weights outperformed applying only to Q and V (the common shortcut). The rank r is the key hyperparameter: $r=4$ is very efficient but lower quality; $r=64$ approaches full fine-tuning quality at $4\times$ the LoRA parameter count.

Q20. What are diffusion models? Explain the forward and reverse processes.

Diffusion models learn to reverse a gradual noising process. Forward process: a Markov chain that progressively adds Gaussian noise to data over T steps: $q(x_t | x_{t-1}) = N(x_t; \sqrt{1-\beta_t} \times x_{t-1}, \beta_t \times I)$. After $T=1000$ steps, $x_T \sim N(0, I)$ (pure noise). Closed-form sampling at any step t : $x_t = \sqrt{\alpha_t} \times x_0 + \sqrt{1 - \alpha_t} \times \epsilon$, where $\epsilon \sim N(0, I)$ and $\alpha_t = \text{product of } (1-\beta_1) \dots (1-\beta_t)$. Reverse process: learn a neural network (U-Net or Transformer) that predicts the noise $\epsilon_{\theta}(x_t, t)$. Training: sample random (x_0, t, ϵ) , compute x_t , train ϵ_{θ} to predict ϵ . Loss: $\text{MSE}(\epsilon_{\theta}(x_t, t), \epsilon)$. Generation: start from $x_T \sim N(0, I)$, iteratively denoise using ϵ_{θ} to reach x_0 .

Real-World Example:

At OpenAI, DALL-E 2 uses a hierarchical diffusion model: a CLIP image encoder maps the text to a semantic embedding; a 'prior' diffusion model generates an image embedding from the text embedding; a 'decoder' diffusion model generates the full 1024×1024 pixel image from the image embedding. The 1000-step reverse process is the generation — each step applies the U-Net to denoise x_t by a small amount. Fast sampling (DDIM) reduces 1000 steps to 50 with similar quality by using a non-Markovian reverse process. Stable Diffusion (Stability AI) applies diffusion in a compressed latent space (VAE encodes 512×512 image to 64×64 latent) — reduces computation by $64\times$ vs pixel-space diffusion. At 50 DDIM steps $\times 64 \times 64$ latent $\times$ U-Net inference: generates an image in 3-5 seconds on a single A100.

HARD — 5 Questions with Answers

Q1. Design a distributed training system for training a 100B parameter language model.

Three-dimensional parallelism required: (1) Data parallelism (DP): split the training batch across D workers. Each worker has a full model copy. Gradients are averaged with AllReduce (Ring-AllReduce) across all DP workers after each step. Communication: $2 \times \text{parameters} \times \text{sizeof(float)} \text{ per AllReduce} = 800\text{GB}$ (too large for a single AllReduce). (2) Tensor parallelism (Megatron-LM): split individual matrices across T GPUs. For attention: each GPU handles a subset of attention heads. For FFN: split the weight matrix row-wise and column-wise. No communication between heads within a layer; one AllReduce per layer ($2 \times \text{activation_size}$). Reduces per-GPU memory by $T \times$. (3) Pipeline parallelism (GPipe): split model depth across P GPUs. Each GPU holds P consecutive transformer layers. Forward pass: micro-batch flows through GPU 0 $\rightarrow 1 \rightarrow \dots \rightarrow P-1$. Backward pass: reversed. 1F1B schedule (interleaved) hides pipeline bubbles to $<5\%$ of compute. Memory: model parameters / $(D \times T \times P)$ per GPU. Gradient checkpointing: recompute activations during backward pass instead of storing them — trade compute for memory.

Real-World Example:

Meta's LLaMA-65B training on 2048 A100 GPUs used 3D parallelism: $D=8$ (data), $T=4$ (tensor), $P=8$ (pipeline) = 256 GPU groups of 8. A single A100 (80GB) holds: $65\text{B params} / 32 = 2\text{B params per GPU} = 4\text{GB (bf16)}$. Gradients = 4GB. Optimizer states (Adam: 2 moments) = 8GB. Activations with grad checkpointing = 20GB. Total $\approx 36\text{GB}$ — fits in 80GB A100 with margin. Training throughput: 380 TFLOPS per GPU (47.5% Model FLOPs Utilisation of A100's 800 TFLOPS). Total compute: $65\text{B} \times 2 \times 1.4\text{T tokens} \times 6$ (FLOPs/token/param for 6ND formula) = 1.1×10^{23} FLOPs. At 380 TFLOPS $\times$ 2048 GPUs = 780 PFLOPS $\rightarrow$ training time = 16 days. The pipeline 1F1B schedule and gradient checkpointing were the two most impactful optimisations: without them, either memory overflow or 30%+ pipeline bubble overhead would have made the run infeasible.

Q2. Implement RLHF from preference data to policy using PPO.

Stage 1 — Supervised Fine-Tuning (SFT): fine-tune the base LLM on high-quality human-written demonstrations using standard cross-entropy loss. Stage 2 — Reward Model (RM) training: collect comparison data: for each prompt, show humans K responses, collect pairwise preferences. Train reward model R_{θ} (same architecture as SFT + linear value head): Bradley-Terry pairwise loss: $L_{\text{RM}} = -\log(\sigma(R_{\theta}(x, y_w) - R_{\theta}(x, y_l)))$ where y_w is preferred, y_l is rejected. Stage 3 — PPO fine-tuning: for each prompt, generate response with policy π_{θ} . Score with R_{θ} . Total reward = $R_{\theta}(x, y) - \beta \times \text{KL}(\pi_{\theta}(y|x) \parallel \pi_{\text{SFT}}(y|x))$ where β prevents reward hacking by penalising large divergence from SFT. PPO objective: $L_{\text{PPO}} = E[\min(r_t \times A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \times A_t)]$ where $r_t = \pi_{\theta}(a_t) / \pi_{\text{old}}(a_t)$ and A_t is advantage (estimated by GAE). Advantage estimation: uses a separate value head (critic) V_{ϕ} trained alongside the policy.

Real-World Example:

Anthropic's RLHF pipeline for Claude: Stage 1 SFT on 50K human demonstrations of helpful, harmless, honest responses. Stage 2 RM: 200K preference comparisons by human annotators. RM accuracy on a held-out comparison set: 74% (models disagree on subtle cases). Stage 3 PPO: 4K RLHF steps from 50K diverse prompts. KL penalty $\beta=0.1$ prevents Claude from diverging too far from the SFT baseline — without it, reward hacking emerges (Claude learns to exploit RM weaknesses: verbosity, sycophancy). Key engineering challenge: RM overoptimisation — as PPO improves the policy, it finds prompts and responses that score high on the RM but are actually poor quality (the RM score diverges from human preference). Anthropic's solution: regularise with a frozen 'gold RM' trained separately, reject responses that score poorly on the gold RM even if the main RM scores them highly. This two-RM approach maintained alignment throughout training.

Q3. Design a multimodal AI system integrating vision and language.

Architecture: (1) Vision encoder: pre-trained ViT (Vision Transformer) produces patch embeddings. 224×224 image $\rightarrow$ 16×16 patches $\rightarrow$ 196 patch tokens. Each 768-dimensional (ViT-Base) or 1024-dimensional (ViT-Large). (2) Text encoder: pre-trained LLM (e.g. LLaMA) with token embeddings. (3) Projection layer: linear or 2-layer MLP mapping vision tokens to LLM's embedding space: $W_{\text{project}} \times [196 \times 768] \rightarrow [196 \times 4096]$. This aligns visual features with the LLM's input space. (4) LLM backbone: receives concatenated [image_tokens, text_tokens] as input. Causal attention mask: text tokens can attend to image tokens; image tokens do not attend to text tokens (they are pre-processed by ViT without text context). Training stages: Stage 1 — freeze LLM, freeze ViT, train only projection layer on (image, caption) pairs — alignment pre-training. Stage 2 — unfreeze LLM (fine-tune), keep ViT frozen, train on (image, instruction, response) triples — instruction tuning.

Real-World Example:

Meta's LLaVA (Large Language and Vision Assistant) follows this exact architecture. Stage 1 (projection training, 595K image-caption pairs, 4 hours on 8 A100s): the projection layer maps CLIP ViT-L/14 features to LLaMA's 4096-dimensional space. Stage 2 (instruction tuning, 158K instruction-following examples, 30 hours on 8 A100s): LLaMA is fine-tuned to respond to visual questions. LLaVA-1.5 (CLIP ViT-L + LLaMA-3-13B): 85.9% on VQAv2 (visual question answering), 90.7% on TextVQA (reading text in images). The key design choice: freeze ViT, only fine-tune LLM. Fine-tuning ViT with language supervision degraded visual feature quality — the CLIP ViT's image-text alignment is better preserved when frozen. Google's PaLI-X (55B parameters) uses a similar architecture but with full end-to-end fine-tuning across both encoders.

Q4. Implement a neural architecture search (NAS) algorithm.

NAS automates the design of neural network architectures. Three components: (1) Search space: define the set of possible architectures. Operation space: {conv3x3, conv5x5, maxpool3x3, avgpool3x3, skip_connection, sep_conv5x5}. Cell-based space: search for the best operations in a cell; stack multiple identical cells. (2) Search strategy: DARTS (Differentiable Architecture Search): represent the architecture as a weighted combination of all possible operations. Use a continuous relaxation: $o_{ij} = \sum_k \text{softmax}(\alpha_k) \times \text{op}_k(x)$. Train architecture parameters α jointly with weight parameters. At the end: discretise by selecting $\text{argmax}_k(\alpha)$ for each edge. Efficient: one training run searches the space. (3) Evaluation strategy: train all architecture candidates on a proxy task (fewer data, fewer epochs), select the best-performing architecture, fully train it. NAS metrics: cell structure performance, parameter count, FLOPs, inference latency.

Real-World Example:

At Google, NAS produced the EfficientNet family. Traditional hand-designed ResNet-50: 4.1M FLOPs, top-1 accuracy 76.0%. EfficientNet-B0 (NAS-discovered baseline architecture): 0.39B FLOPs, 77.1% top-1 — 10× fewer FLOPs and higher accuracy. EfficientNet-B7 (compound scaling of B0): 66B FLOPs, 84.4% top-1. The NAS search found that the optimal cell structure used mobile inverted bottleneck convolutions (MBConv) with squeeze-and-excitation, a design no human had tried before. DARTS was used for the cell structure search (2 GPU-days to find the cell), compound scaling (layer/channel/resolution trade-off) was searched with a small proxy experiment. The NAS-discovered B0 architecture was then manually scaled up to B1-B7 for different compute budgets. EfficientNets became the backbone for Google Photos, Google Lens, and Google Translate image processing.

Q5. Implement a retrieval-augmented generation (RAG) system.

RAG combines a retrieval system with a generative LLM to answer questions using external knowledge. Components: (1) Document ingestion: split documents into chunks (512 tokens). For each chunk: compute embedding using a bi-encoder (sentence-transformers, e.g. all-MiniLM-L6-v2). Store embedding + chunk text in a vector database (Faiss, Pinecone, Weaviate). (2) Query processing: embed the query with the same bi-encoder. Retrieve top-k (k=3-10) chunks by cosine similarity from the vector DB. Optionally re-rank retrieved chunks with a cross-encoder (slower but more accurate: cross-encoder sees both query and document simultaneously). (3) Generation: construct a prompt: 'Context: [chunk1] [chunk2] [chunk3]. Question: [query]. Answer:'. Feed to LLM (GPT-4, LLaMA, etc.). LLM generates the answer grounded in the retrieved context. (4) Evaluation: faithfulness (does the answer contradict the retrieved context?), relevance (does the retrieved context contain the answer?), answer quality.

Real-World Example:

At Meta, their internal code assistance tool uses RAG over their 100M+ line codebase. The retrieval: every function in the codebase is indexed as a chunk (function body + docstring). Query 'How does the user authentication work?' retrieves the top-10 most relevant code chunks from auth.py, permissions.py, and login_handler.py. The LLM (CodeLlama) generates an explanation grounded in the actual production code, not outdated documentation. Without RAG: the LLM would hallucinate function signatures or describe generic authentication patterns not specific to Meta's implementation. With RAG: 78% of engineer-rated responses were factually accurate (vs 31% without RAG). The vector index is updated nightly — new functions are indexed, deleted functions are removed from the vector DB. Meta's RAG system processes 50K developer queries per day.